
RTInfer: Real-Time Inference of Multiple DNNs on Edge GPUs

Renjie Li¹ Tong Sun¹ Yi Gao¹ Wei Dong¹

Abstract

While edge GPUs are increasingly used for latency-critical DNN tasks, limited resources often fail to meet strict real-time (RT) requirements under concurrent workloads. Existing preemption and early-exit mechanisms often underutilize GPU resources through single-task queuing and sacrifice excessive accuracy during task bursts. To address this, we propose RTInfer, a novel system that enables concurrent RT task execution while balancing throughput and accuracy. RTInfer integrates an accuracy-calibrated lightweight variant co-optimization to generate efficient models, a memory-layout-aware scheduler to mitigate fragmentation during preemption, and an on-demand loading strategy to minimize host-to-GPU latency. Extensive evaluations demonstrate that RTInfer outperforms state-of-the-art methods by reducing average deadline miss rate (DMR) from 32.8% to 0% and improving accuracy by up to 56.5%.

1. Introduction

Deep learning (DL) has emerged as a transformative technology for a variety of edge-based applications (Zhao et al., 2021; Ling et al., 2021; Liu et al., 2023), including augmented reality and autonomous vehicles (AV). These applications often require the simultaneous execution of multiple deep neural network (DNN) tasks, each varying in criticality. For example, AV utilize DNNs to perform real-time (RT) tasks, such as obstacle detection and traffic light recognition (Jang et al., 2020). In contrast, other tasks that do not have strict real-time requirements are classified as best-effort (BE) tasks, such as driver monitoring and in-car entertainment. To accelerate the execution of DL applications, GPUs are increasingly used to run DNN tasks on emerging edge devices, such as NVIDIA Jetson series (NVIDIA, 2024).

¹College of Computer Science, Zhejiang University, Hangzhou, Zhejiang, China. Correspondence to: Yi Gao <gaoyi@zju.edu.cn>.

Edge devices often encounter bursts of concurrent RT tasks with diverse requirements. For instance, an autonomous vehicle must simultaneously process vision tasks from front, side, and rear-view cameras, with deadlines ranging from 150ms to 500ms (Xiang & Kim, 2019). These tasks demand high inference accuracy to prevent catastrophic failures in safety-critical scenarios.

However, existing approaches cannot meeting the real-time and accuracy requirements of multiple RT tasks in a burst manner. REEF (Han et al., 2022) and Miriam (Zhao et al., 2023) both employ preemption mechanisms (RT tasks preempting BE tasks). However, multiple RT tasks are executed in the order of their arrival, causing subsequent RT tasks to miss their deadlines due to waiting. A recent advancement, Pantheon (Han et al., 2024), enables RT tasks to preempt each other based on deadline urgency and utilizes early exits to meet timing constraints. While effective under light loads, Pantheon’s strict serial execution of RT tasks leaves GPU resources underutilized and forces significant accuracy sacrifices to satisfy deadlines during task bursts. Consequently, subsequent RT tasks must still wait for the current one to complete, even when available resources could support concurrent execution.

To overcome the resource underutilization caused by serial RT execution, we propose RTInfer, which supports parallel inference for multiple RT tasks. However, enabling concurrent RT execution introduces three unique challenges:

- *Concurrent inference of multiple RT tasks can easily exhaust the limited GPU memory budget, while naive pruning to alleviate memory pressure can substantially degrade model accuracy.*
- *Scheduling tasks solely based on their temporal urgency can easily lead to memory fragmentation, degrading concurrent performance.*
- *Host-to-GPU on-demand loading during variant switches can dominate RT latency and cause deadline misses.*

To address this challenge, we present **RTInfer**, which enables concurrent inference of multiple real-time DNNs on edge GPUs while jointly controlling memory, latency, and accuracy. Instead of aggressive pruning (Fang et al., 2023), RTInfer treats pruning as a controlled design dimension

to reduce memory footprint and applies offline fine-tuning to accuracy-calibrate pruned and early-exit variants. This avoids the severe accuracy loss commonly observed in naive pruning-based approaches. To further cope with limited GPU execution resources, RTInfer integrates early-exit mechanisms to construct a spectrum of model variants with different pruning ratios and exit depths, enabling flexible online selection. Finally, we perform multi-objective co-optimization over memory, latency, and accuracy to retain only high-quality variants, preventing excessive deployment overhead and search-space explosion.

Another challenge stems from urgency-based scheduling for real-time tasks (Han et al., 2024), which can induce severe memory fragmentation. We observe that inefficient memory layouts limit real-time concurrency, forcing tasks to exit prematurely and sacrifice accuracy to meet deadlines. To address this issue, we propose a memory-layout-aware scheduling method. Upon each task arrival, the scheduler first prioritizes queued tasks via deadline-driven urgency, and then applies lightweight heuristics to jointly optimize execution order and memory layout for the active task set.

Beyond memory capacity and layout, host-to-GPU on-demand loading becomes a critical bottleneck when many variants of the same DNN coexist. Naively loading full variants on demand incurs excessive transfer latency, while preloading all variants exhausts GPU memory. We therefore design a load-aware pipeline based on Delta-Graph with shared-segment deduplication. Offline, variants are repackaged as a shared trunk with sparse deltas and packed into page-aligned, content-addressed chunks. Online, the scheduler prefetches only missing chunks to meet first-use deadlines and selects zero-copy reads or materialize-once merges to preserve kernel efficiency.

Evaluation. We develop a prototype of RTInfer and evaluate its performance on a prevalent edge device (NVIDIA Xavier Jetson NX) across a broad range of DL applications. Experiments show that RTInfer outperforms the classical task scheduling methods in real-time areas and the state-of-the-art (SOTA) multi-DNN scheduling Pantheon (Han et al., 2024), reducing average task deadline miss rates (DMR) from 66.6% and 32.8% to 0%, respectively. Meanwhile, RTInfer can enhance the accuracy performance by $2.2\text{--}6.3\times$ and $34.5\text{--}56.5\%$, respectively. Additionally, RTInfer is lightweight that only taking $\sim 34.6\ \mu\text{s}$ for each scheduling.

Contributions are summarized as following:

- We propose a novel design, exploiting concurrency for multiple real-time DNN inference on edge GPUs to meet real-time task deadlines while maintaining high accuracy.
- We address unique challenges in RTInfer with several novel techniques, i.e., accuracy-calibrated lightweight variant co-optimization method, memory-layout-friendly

task scheduling and delta-graph and load-aware pipeline. The prototype implementation of RTInfer has been released as open source and is available at <https://github.com/ELE-Jay/RTInfer>.

- We develop a prototype system of RTInfer and evaluate it with representative multi-DNN RT tasks and edge devices. The results show significant performance gains compared to the SOTA scheduling design on edge GPUs.

2. Related Work

Multi-DNN inference scheduling on GPUs. Traditional real-time scheduling methods on edge GPUs often fall short for DNN tasks due to a lack of consideration for DNN workload and GPU architecture specifics. REEF (Han et al., 2022), the first DNN inference serving system for commodity cloud GPUs, introduced mixed-criticality multi-DNN inference coordination and kernel-level preemption. Miriam (Zhao et al., 2023) builds upon REEF by incorporating an elastic kernel for fine-grained multi-stream processing on edge GPUs, significantly enhancing parallelism across multiple DNNs. Despite these advancements, timely task execution in multi-tasking scenarios remains a challenge, and maintaining real-time performance is still an issue. To improve inference efficiency, adaptive neural networks (Bolukbasi et al., 2017) learn to selectively evaluate model components for different examples, enabling early exits to reduce computational cost. Similarly, Shallow-Deep Networks (Kaya et al., 2019) characterize the “overthinking” phenomenon and introduce internal classifiers to off-the-shelf DNNs, allowing confidence-based early exits that preserve accuracy. CoDL (Jia et al., 2022) proposes hybrid-type data sharing and lightweight latency prediction to partition operators between CPU and GPU, reducing data transfer overhead and improving single-model inference efficiency. NN-Stretch (Wei et al., 2023) automatically transforms sequential models into multi-branch architectures for parallel execution across heterogeneous processors. While effective for single-model acceleration, these methods do not address inter-task resource contention or memory constraints when multiple real-time DNNs compete for limited on-device resources. Leveraging these dynamic inference theories, A recent progress Pantheon (Han et al., 2024) achieves preemptible multi-DNN inference through early model exits. However, the limited memory of edge devices prevents the concurrent execution of multiple real-time tasks, and the lack of consideration for memory layout during scheduling leads to unnecessary early exits and subsequent accuracy degradation.

Memory-constrained inference. Previous work (Huang et al., 2020; Peng et al., 2020) reduces GPU memory requirements by swapping with main memory on cloud or edge servers. However, due to the significant latency intro-

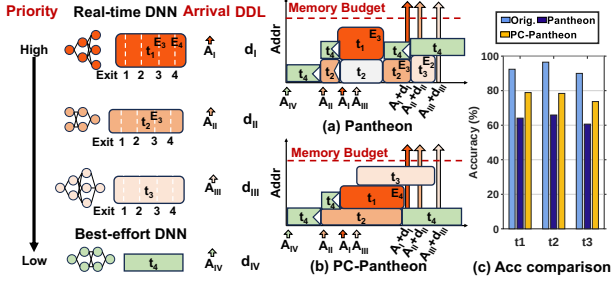


Figure 1. Illustration of SOTA multi-DNN scheduling and enhanced method. We compare accuracy of RT tasks under two methods with original models.

duced by swapping, these approaches might cause missed deadlines for real-time tasks. Heimdall (Yi & Lee, 2020) mitigates GPU resource contention in AR by decomposing large DNNs into smaller units via pseudo-preemption, prioritizing latency-sensitive rendering tasks to stabilize memory bandwidth and enhance utilization on mobile GPUs. However, it lacks explicit memory layout optimization, risks fragmentation, and incurs memory bandwidth overhead due to frequent context switching. Melon (Wang et al., 2022) reduces the memory footprint for on-device training through offline planning with a lifetime-aware memory pool. FlexNN (Li et al., 2024) achieves optimal memory utilization and minimal memory management overhead through a slicing-loading-computing joint planning approach. These methods do not consider the preemption between real-time tasks or the joint optimization of task priority scheduling and memory layout, potentially leading to suboptimal performance in our scenario. RTInfer introduces a scheduling approach that jointly considers memory layout and real-time task priorities to achieve a lower DMR.

3. System Model and Motivation

Task Model. We consider a set of N real-time DNN tasks $\mathcal{T} = \{T_i\}$, categorized into: (i) **Real-time tasks** with strict deadlines D_i and inter-arrival times P_i that can preempt each other; and (ii) **Best-effort tasks** with lower priority handled via FIFO (Zhao et al., 2021; Ling et al., 2021). Each job of T_i selects a variant $v \in \mathcal{V}_i$ (generated via offline pruning and early-exit insertion) with pre-measured latency $L_{i,v}$, peak GPU memory $M_{i,v}$, and accuracy $A_{i,v}$.

Execution and Metrics. Jobs run on a single edge GPU supporting concurrent kernels and cooperative preemption (Zhao et al., 2023; Han et al., 2024). We model each buffer k as a time-address rectangle $b_k = [t_k^{\text{start}}, t_k^{\text{end}}] \times [a_k, a_k + \text{size}_k]$ in a 2D "time $\times$ memory" space, where S denotes the current schedule state. System performance is quantified by the **Deadline Miss Ratio (DMR)**, defined as the ratio of missed deadlines to total jobs. Jobs missing their

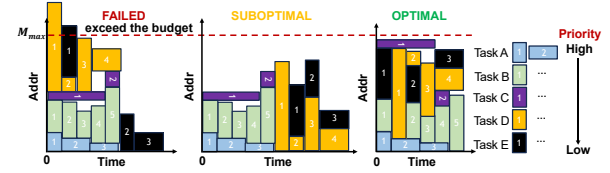


Figure 2. Different memory management strategies.

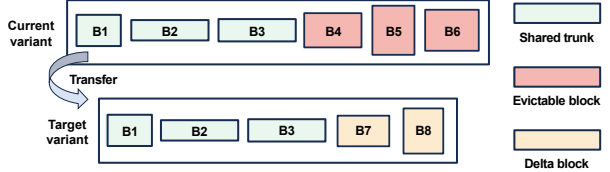


Figure 3. Delta-Graph based variant switching and chunk reuse.

deadlines contribute zero to the relative accuracy metric.

Limitations of SOTA Scheduling. Existing edge GPU schedulers prioritize real-time (RT) tasks via preemption mechanisms. As shown in Fig. 1(a), the state-of-the-art (SOTA) system Pantheon (Han et al., 2024) employs model slicing and nested redundancy to enable RT tasks to preempt each other based on urgency. However, our experiments on an NVIDIA Jetson Xavier NX (Fig. 1(c)) reveal that Pantheon’s serial RT execution and heavy reliance on early exits cause significant accuracy degradation (up to 30.6%) as task density increases. To meet deadlines, it dynamically switches to smaller model variants via early exits.

Insight 1: while executing multiple RT tasks concurrently could improve throughput and reduce early-exit reliance, naive concurrency quickly exhausts the limited GPU memory budget. As illustrated in Fig. 1(b), applying model pruning (PC-Pantheon) helps but introduces irreversible capacity loss, leading to a 13.5%–18.1% accuracy drop. *Therefore, enabling RT concurrency requires a joint optimization of pruning ratios and early-exit depths to balance memory constraints with accuracy preservation.*

Insight 2: strict priority-based scheduling often leads to suboptimal memory layouts. As shown in Fig. 2, allocating memory for Task D before Task E can trigger overflow due to fragmentation, even if total capacity is sufficient. Conversely, prioritizing the lowest-priority Task E’s layout first (Right of Fig. 2) ensures all tasks fit. *Therefore, effective scheduling must jointly optimize task priority and spatial memory layout to prevent unnecessary early exits.*

Insight 3: switching between many DNN variants triggers significant host-to-device (H2D) transfer latency (e.g., $\approx 83.4\text{ms}$ for 1GB), dominating the inference window and causing deadline misses. Preloading all variants is impractical as it exhausts memory. As shown in Fig. 3, sharding weights into a shared trunk and sparse deltas enables *delta loading*, where only missing blocks move during switches.

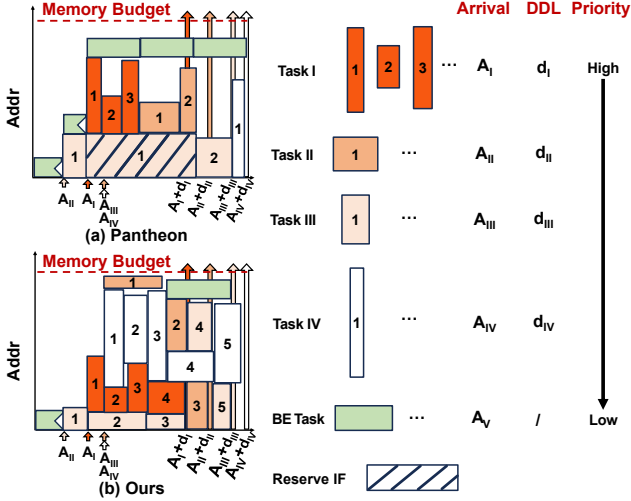


Figure 4. Illustration of SOTA multi-DNN scheduling and RTInfer design.

Therefore, the scheduler must explicitly bound H2D loading costs before first-use to maintain real-time guarantees.

Figure 4(b) illustrates our proposed system, RTInfer, which integrates the aforementioned insights. By generating accuracy-calibrated lightweight variants, RTInfer enables resource-constrained edge GPUs to serve a significantly higher number of concurrent RT tasks without sacrificing precision. While maintaining urgency-based prioritization similar to Pantheon under low load, RTInfer dynamically adapts to high-density bursts by mitigating memory fragmentation and enforcing load-aware variant switching. These joint optimizations allow RT tasks to delay or even bypass early exits, achieving superior inference accuracy while strictly meeting real-time deadlines.

4. System Design

RTInfer consists of an offline preprocessing stage for variant pool construction and an online scheduling stage for concurrent inference (Fig. 5).

4.1. Accuracy-Calibrated Lightweight Variant Co-Optimization

To satisfy concurrent RT tasks under strict memory budgets, we jointly optimize pruning ratios P and early exits E to construct a high-quality variant space $V = \{(p, e) \mid p \in P, e \in E\}$.

While extensive works (Wan et al., 2023; Fang et al., 2023; 2024b) have focused on adaptive model pruning, applying these methods to early-exit architectures remains non-trivial. First, early exits function as premature output layers whose layer-wise dependencies must be pruned in synchronization with the backbone. Second, pruning must preserve the

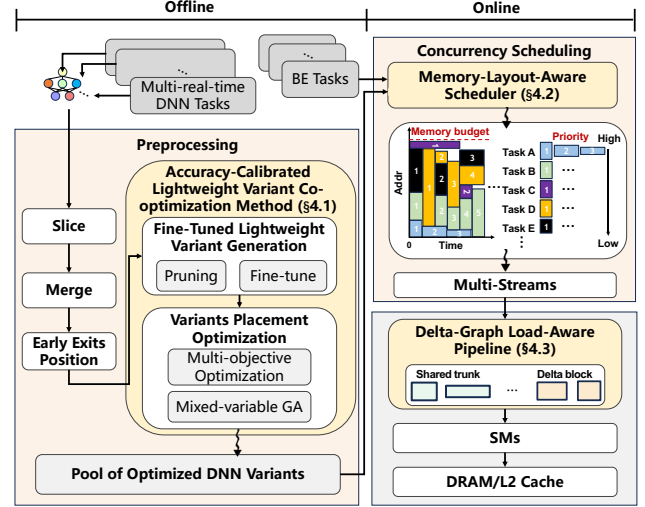


Figure 5. Overview of the RTInfer design.

tensor interfaces between model chunks to ensure seamless intermediate result transmission. Third, the coupled dependencies between multiple exit heads and backbone structures must be carefully assessed to prevent accuracy collapse from pruning critical parameters. To address these, we propose an accuracy-calibrated lightweight variant co-optimization method that jointly prunes and fine-tunes the backbone and exit heads while maintaining structural consistency. A schematic diagram of our pruning method is shown in Figure 6. **Step 1** prunes the initial chunk and exit while preserving output interfaces. **Steps 2–3** freeze preceding chunks to ensure stable intermediate result transmission. This strategy explicitly assesses layer dependencies and weight importance to prevent accuracy collapse. Detailed pruning challenges and multi-step logic are provided in Appendix A.1.

Placement Optimization. We trade off latency L , memory M , and accuracy A via:

$$\min_{E, P} L(E, P) + \lambda_1 M(E, P) + \lambda_2 (1 - A(E, P)) \quad (1)$$

subject to $\Delta a \leq \alpha$ (accuracy cap) and $M \leq \beta$ (memory budget). We solve Eq. 1 using a **mixed-variable Genetic Algorithm (GA)**. *Design Justification:* GA is chosen as standard solvers fail in our non-smooth space: Grid Search suffers from combinatorial explosion, while Bayesian Optimization struggles with the discontinuous feasibility boundaries induced by memory layout constraints (see Appendix A.2 for comparative analysis).

4.2. Memory-Layout-Aware Task Scheduling

Upon task arrival, the scheduler sorts the queue by urgency. The memory allocator then assigns each buffer $B \in \mathbb{N}^3$ a starting address pos_i within budget M (Kim et al., 2023;

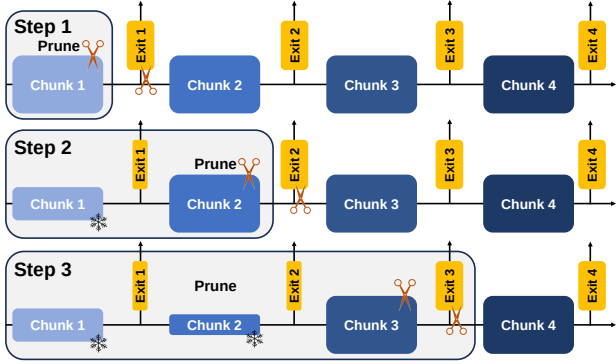


Figure 6. Pruning for DNN chunks and early exits.

Ghafari & Mansouri, 2023; Dehghankar et al.). To ensure validity, the allocation must satisfy: (i) $pos_i \in \mathbb{Z}^+$, (ii) no two active buffers overlap in the 2D “time $\times$ memory” plane, and (iii) the maximum address $pos_i + size_i \leq M$. To mitigate fragmentation, we draw inspiration from prior work on optimization-based memory allocation (Maas et al., 2023) and formulate memory-layout-aware scheduling as an integer linear programming (ILP) (Lu et al., 2024; Ramesh et al., 2024; Ahmed et al., 2024) problem, as shown in Figure 7. We use $pos_i \in \mathbb{N}$ to encode the position of the i -th buffer’s lower end. Since the start and end times are fixed, we can statically determine which buffers overlap in the time dimension. For each pair of overlapping buffers (*OVLBuffers*), we add the constraints to encode that they must not overlap in the spatial dimension as follows:

- s.t. (1) $\forall_{i \in Buffers}, pos_i + size_i \leq M$.
 (2) $\forall_{i,j \in OVLBuffers}, B_{i,j} + \tilde{B}_{i,j} = 1$.
 (3) $\forall_{i,j \in OVLBuffers}, pos_i + size_i - B_{i,j} \times M < pos_j$.
 (4) $\forall_{i,j \in OVLBuffers}, pos_j + size_j - \tilde{B}_{i,j} \times M < pos_i$.

For two buffers i and j , this can be represented by $(pos_i + size_i < pos_j) \vee (pos_j + size_j < pos_i)$. We use a standard ILP solver, which encodes it as Boolean variables, i.e., $B_{i,j} \vee \tilde{B}_{i,j}$.

We solve this via a **Heuristic-driven Search** for ILP (Alg. 1 in Appx) prioritizing blocks by *Area* ($size \times lifetime$), *Size*, and *Lifetime*. If a placement violates a deadline, RTInfer triggers an immediate early exit. We employ **Minor Backtracking** (swapping candidates at the current level) and **Major Backtracking** (returning to the root conflict node) to efficiently navigate the search tree under extreme memory pressure (see Appendix A.3 for backtracking logic).

4.3. Delta-Graph and Load-Aware Pipeline

With many variants of the same model, switching variants online can trigger large host-to-device transfers that compete with RT deadlines. We conservatively upper-bound the

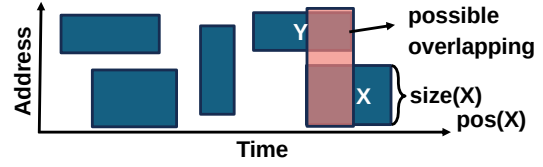


Figure 7. The ILP formulation encodes the position of each buffer as a variable and incorporates constraints to prevent any two buffers from overlapping.

transfer time of size S by

$$T_{load} = t_0 + \frac{S}{\beta_{min}}, \quad (2)$$

where β_{min} is a measured bandwidth floor and t_0 is a small startup cost.

Offline. Delta-Graph represents all variants as a *shared trunk* plus *sparse deltas*. Weights are *tiled along channels* and packed into *page-aligned chunks* (1–4 MB). Chunks are *content-addressed*: identical content across variants (or models) is stored once and, when loaded, held once in GPU memory as read-only with reference counting. For each variant (p, d) we record $S(p, d)$, the sum of bytes of chunks not already resident, which is the quantity that determines T_{load} .

Online. When a variant is selected, the runtime computes the set of *missing* chunks and issues prefetches. For each required chunk c , the scheduler enforces a *first-use* constraint

$$t_c^{load, end} \leq s_l^{first-use}, \quad (3)$$

ensuring the chunk arrives before layer l consumes it. To balance write overhead and kernel throughput, we offer two execution paths: (i) *zero-copy (indirection)*, which reads trunk+ Δ chunks via a mapping without extra writes (suited for low-reuse layers); and (ii) *materialize-once merge*, which assembles deltas into a contiguous buffer once so that vendor kernels (cuDNN/cutlass) run at full speed (suited for hot layers). The memory-layout-aware scheduler packs both compute buffers and transfer chunks on the time $\times$ address plane; shared chunks occupy a single address and are referenced by multiple tasks. Admission rejects or auto-downgrades to lighter variants if slack cannot cover $\sum_c T_{load}(S_c)$ plus compute.

Variant selection. We extend the 2D Variant Atlas (pruning $p \times$ early-exit depth d) with $S(p, d)$ and retain a sparse Pareto backbone. Online selection is $O(1)$ and jointly considers latency, memory, accuracy, and bytes moved on switch. A small set of precompiled kernel configurations per hot layer can be reused across nearby variants to reduce tuning latency, but they are ancillary to the load-aware design.

Table 1. Details of the three applications used for evaluation. The setup contains fifteen periodic real-time DNN streams. Period and deadline are the base values before deadline scaling by κ . Accuracy denotes the full-depth unpruned model profile.

Application	Stream	Task Type	DNN Model	Dataset	Period (ms)	Deadline (ms)	Accuracy (%)
Smart traffic	T1	High-res object detection	YOLOv8-L	KITTI (Geiger et al., 2012)	180	320	92.40
	T2	High-res object detection	YOLOv8-L	KITTI	180	320	92.40
	T3	High-res object detection	YOLOv8-L	KITTI	220	340	92.40
	T4	Traffic alert detection	YOLOv8n	KITTI	320	180	89.60
	T5	Traffic alert detection	YOLOv8n	KITTI	360	210	89.60
UAV ground station	T1	Large scene recognition	ViT-L	Scene-15 (Fei-Fei & Perona, 2005)	240	500	96.50
	T2	Large scene recognition	ViT-L	Scene-15	260	500	96.50
	T3	Large scene recognition	ViT-L	Scene-15	300	540	96.50
	T4	Emergency scene classification	MobileViT-S	Scene-15	360	220	95.10
	T5	Emergency scene classification	MobileViT-S	Scene-15	420	240	95.10
Service robot	T1	Navigation perception	YOLOv8-L	KITTI	190	320	92.40
	T2	Navigation perception	YOLOv8-L	KITTI	230	340	92.40
	T3	Transformer scene summary	ViT-L	Scene-15	360	270	96.50
	T4	Edge command generation	GPT-2 Small	Edge command prompts	420	220	90.00
	T5	Edge command generation	GPT-2 Small	Edge command prompts	500	240	90.00

5. Implementation and Testbed

Implementation. Implementation details are provided in Appendix B. RTInfer is implemented as an event-driven runtime for multiple real-time DNN inference streams on edge GPUs. Each RT task is associated with a high-priority execution stream, while best-effort (BE) tasks run on low-priority streams and can be paused or placed in remaining memory gaps. The CPU-side scheduler is triggered by job releases and completions, and it jointly selects model variants and their time-address memory placement. The runtime is built on CUDA priority streams and manages GPU buffers in user space through CUDA driver APIs (CUDA 11.2; `cudaMallocAsync` with a custom memory pool), which is the mechanism used to realize the time-address placement decisions.

The online runtime performs memory-layout-aware placement in an event-driven loop triggered on job release and completion, avoiding per-frame overhead. Concurrency stems from CUDA’s concurrent kernel execution across SMs. By keeping the live buffers of co-resident kernels non-overlapping in the device address space, our allocator enables the concurrent execution of RT tasks whenever SM resources and memory constraints are jointly satisfied.

To avoid per-job churn, variants are warmed once and pinned when admitted. Eviction proceeds in least-recently-used (LRU) order, reclaiming BE variants first, then RT variants with the largest slack.

Testbed. Online components run on an NVIDIA Jetson Xavier NX (384-core Volta GPU) with 8 GiB unified memory. Since OS services, drivers, sensor runtime, CUDA allocator slack, and other edge software consume part of this memory, we use a 6144 MiB effective scheduling budget in the main evaluation. offline variant generation uses a host with an Intel i9-10900K CPU and an RTX 3090 Ti GPU. The offline pipeline uses Python 3.8 and PyTorch (with Torch-Pruning) to construct variants; the online runtime is

implemented in C++17 with CUDA 11.2 and LibTorch.β

6. Evaluation

6.1. Experiment Setup

Multi-DNN tasks.

Following previous works (Han et al., 2024; Zhao et al., 2023), we evaluate RTInfer on DNN streams from three representative edge applications, while preserving the original application categories but replacing the previous CNN-only task mixes with modern and larger models:

- **Smart Traffic:** YOLOv8-L (Jocher et al., 2023) with 1080p inputs and YOLOv8n with 640×640 inputs for traffic detection.
- **UAV Ground:** ViT-L (Dosovitskiy et al., 2020) with 1024×1024 inputs and MobileViT-S (Mehta & Rastegari, 2021) with 512×512 inputs for scene recognition.
- **Service Robot:** YOLOv8-L with 1080p inputs, ViT-L with 1024×1024 inputs, and an edge GPT-2 Small (Radford et al., 2019) command-generation task with bounded KV-cache memory.

Repeated DNN entries in the Table 1 denote independent sensor streams or service requests sharing the same model architecture. In implementation, RTInfer stores one shared copy of resident model-weight chunks whenever variants overlap, while each live inference job keeps separate input, activation, workspace, and intermediate-feature buffers. Thus, memory pressure in repeated streams mainly comes from per-stream runtime buffers rather than duplicated model weights. For workloads, the offline ALC stage enumerates pruning tiers $\{0, 0.25, 0.5, 0.75\}$ and all early-exit points, then retains variants whose pruning-induced accuracy drop is within the constraint parameter $\alpha = 5\%$. This α bounds the degradation introduced by pruning at a given

exit; the accuracy drop caused by choosing an earlier exit is reported explicitly in the completed-only and deadline-weighted accuracy metrics. The memory constraint parameter β in C_{mem} is set to the effective Jetson scheduling budget, i.e., $\beta = 6144$ MiB in the main Xavier NX evaluation rather than the raw 8 GiB physical DRAM capacity. Smart Traffic requires 6500 MiB, UAV Ground requires 7070 MiB, and Service Robot requires 7950 MiB. All three exceed the 6144 MiB effective Jetson budget before adding BE work, H2D staging buffers, CUDA allocator fragmentation, or preempted intermediate-feature state. Each application contains five periodic real-time streams with staggered release offsets and heterogeneous periods/deadlines, as listed in Table 1. This setup captures diverse timing requirements across real-time DNN tasks. For deadline-scaling experiments, we uniformly multiply the deadlines in Table 1 by κ , while keeping periods and release offsets unchanged. Each application also includes a best-effort background analytics workload on a low-priority CUDA stream, which can run concurrently with RT tasks when sufficient time-address space is available.

Baselines. We compare RTInfer with the following methods:

- **RMS-P** (Kang et al., 2021): applies rate-monotonic scheduling to pruned and fine-tuned DNN models, prioritizing tasks with shorter execution times.
- **DMS-P** (Xiang & Kim, 2019): applies deadline-monotonic scheduling to pruned and fine-tuned DNN models, prioritizing tasks closer to their deadlines.
- **Pantheon** (Han et al., 2024): a state-of-the-art system for multi-DNN inference scheduling on edge GPUs.

Metrics. We evaluate each method using two metrics:

- **Deadline Miss Ratio (DMR):** the fraction of jobs that miss their deadlines among all executed jobs.
- **Model accuracy:** reported as relative accuracy, defined as the ratio between a variant’s accuracy and that of the original model in Table 1. Jobs missing deadlines contribute zero accuracy.

6.2. Overall Performance

We evaluate overall performance under varying deadline tightness. Figure 8 reports DMR and accuracy across three applications while scaling all task deadlines by $\kappa \in \{0.6, 0.8, 1.0, 1.2, 1.4\}$. Across all settings, RTInfer achieves the lowest DMR and highest accuracy, with performance gaps varying with deadline tightness.

Tight deadlines ($\kappa = 0.6$ – 0.8). Under tight deadlines, all baselines degrade sharply: RMS-P and DMS-P incur

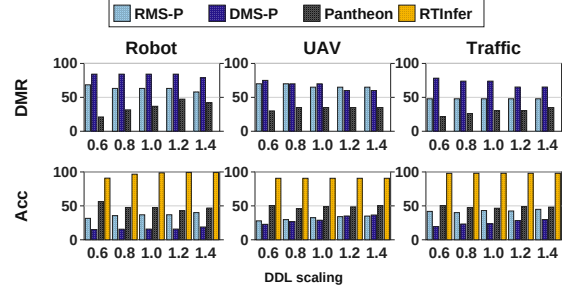


Figure 8. Performance of 4 methods under various applications while uniformly scaling all task deadlines by a factor $\kappa \in \{0.6, 0.8, 1.0, 1.2, 1.4\}$. Top: DMR. Bottom: accuracy.

extremely high average DMRs of 61.2% and 77.6%, respectively, and Pantheon still suffers 27.6% DMR due to limited RT concurrency and costly variant switching. In contrast, RTInfer maintains 0% DMR across all applications, with an average accuracy of 94.2%, benefiting from memory-aware placement, sliced multi-variant inference, and Delta-Graph-based load-aware pipelining that bounds interference and switching overhead.

Moderate deadlines ($\kappa = 1.0$). At default deadlines, performance improves across all methods, yet RTInfer remains superior, keeping DMR below at 0% with 95.8% average accuracy. Its advantage grows with increasing RT concurrency across applications, reflecting effective exploitation of parallelism under bounded GPU memory.

Loose deadlines ($\kappa = 1.2$ – 1.4). When deadlines are relaxed, performance gaps narrow: Pantheon approaches low DMR and high accuracy, while RTInfer retains a modest lead. This trend indicates that RTInfer’s benefits are most pronounced under tight real-time and memory constraints, and diminish gracefully as resources become ample.

6.3. In-depth Analysis

To understand why RTInfer outperforms existing methods under high concurrency, we analyze task-level timeliness and resource efficiency.

Task-level Timeliness. Figure 9 compares the response times of concurrent tasks in the revised service robot application. Pantheon’s serial RT execution queue causes frequent deadline violations for “Cmd” and noticeable response-time spikes for “Nav”. In contrast, RTInfer keeps both tasks within their deadlines by concurrently executing accuracy-calibrated variants under memory-aware placement. This demonstrates the effectiveness of RTInfer’s concurrent scheduling in reducing queuing delay under bounded GPU memory.

Resource Utilization. Figure 10 illustrates the GPU utilization under varying RT-task counts. RTInfer achieves higher and more stable utilization because it supports parallel RT

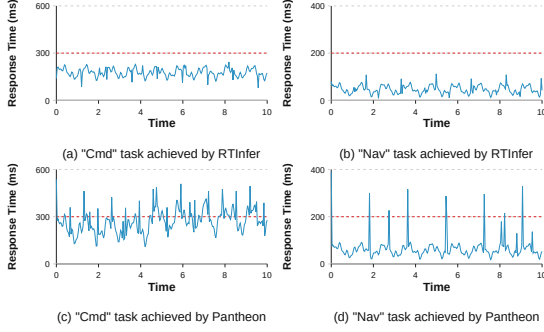


Figure 9. Response time of “Cmd” and “Nav” tasks when concurrently executing the revised service robot application. “Cmd” denotes edge command generation with GPT-2-small KV, and “Nav” denotes navigation perception with YOLOv8-L-1080p.

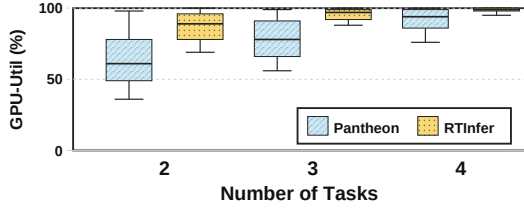


Figure 10. GPU utilization comparison between Pantheon and RTInfer under different task counts.

execution, whereas Pantheon is limited to a single RT-task queue. Further micro-benchmarks on variant design spaces and early-exit accuracy are provided in Appendix C.

6.4. Ablation Study

To further characterize the contribution of each component, we evaluate three key components of RTInfer by comparing it against downgraded versions: (i) **RTInfer-w/o-ALC**: disables the Accuracy-calibrated Lightweight Variant Co-optimization. This version handles tasks with a single, fixed model, leading to parallel execution failure under heavy load, (ii) **RTInfer-w/o-MS**: disables the Memory-layout-aware Scheduler. Inefficient memory management leads to higher fragmentation, breaking the bounded-memory execution guarantees, and (iii) **RTInfer-w/o-DLP**: disables the Delta-Graph and load-aware Pipeline. This version transfers full parameter blocks on each variant switch, making the switching delay unpredictable.

As shown in Figure 11, removing ALC elevates the DMR by about 2 $\times$, as the system loses the ability to trade computation and model capacity for timeliness. Disabling MS increases the DMR by 2–3 $\times$ due to memory fragmentation and less effective bounded-memory placement. Finally, omitting DLP elevates the DMR by 3–6 $\times$, as full parameter

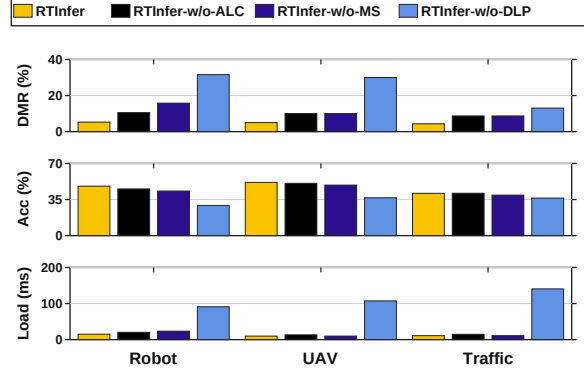


Figure 11. Ablation study across different versions of RTInfer.

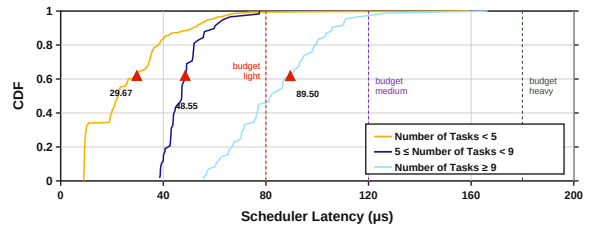


Figure 12. CDF of latency of executing the scheduler algorithm once.

transfers appear on the critical path during variant switching. This demonstrates the importance of RTInfer’s comprehensive approach to optimizing model efficiency, memory management, and load-aware execution for real-time DNN inference.

6.5. System Overhead

In this subsection, we analyze the overheads of RTInfer, including scheduling latency and power consumption. Since RTInfer frequently invokes scheduling with preemption and memory-layout decisions, excessive scheduling latency may delay task execution and cause deadline misses. We therefore bound each scheduler invocation with load-dependent decision budgets.

Scheduling latency. Figure 12 shows the CDF of RTInfer’s scheduling latency. The average latency is 29.67 μ s with fewer than five active tasks and 48.55 μ s with 5–8 active tasks. Even with nine or more active tasks, the average latency remains only 89.50 μ s, well below the 180 μ s heavy-load scheduler budget.

Power consumption. We further evaluate device power consumption using a POWER-Z KM003C power monitor. As shown in Figure 13, the device consumes about 6.40 W on average when idle. Pantheon consumes 8.80 W on average during runtime, while RTInfer consumes 9.56 W on average due to the higher hardware activity introduced by

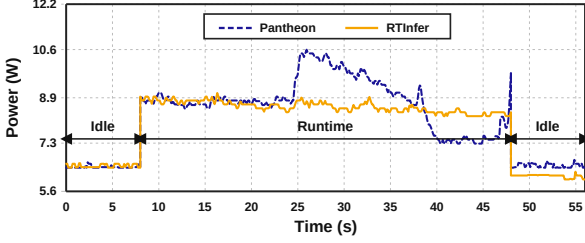


Figure 13. Power consumption comparison.

concurrent DNN execution. These results show that RTInfer improves concurrency and deadline satisfaction with only a modest power increase. Although RTInfer performs more sophisticated online scheduling, this overhead is at the microsecond scale and is negligible compared with DNN execution. The higher average runtime power mainly comes from keeping more DNN work active in parallel. Toward the end of the runtime window, the two curves partially overlap because Pantheon also reaches a burst-heavy steady state under sustained serial DNN execution, driving the device close to a similar peak power level.

7. Discussion and Future Work

While our main deployment setting is edge DNN inference, RTInfer is architecturally agnostic and can support both CNN- and Transformer-based models (Xu et al., 2023; Ma et al., 2023a). Our pruning-based approach is orthogonal to quantization (Han et al., 2015), allowing for joint application to further reduce memory footprint. RTInfer is most effective for memory- and loading-bound real-time DNN workloads, while its benefits may be limited for SM-saturated or large-kernel workloads, motivating the SM-aware extensions discussed in Appendix D. Currently, RTInfer treats DNN tasks independently due to task heterogeneity (e.g., in AV systems); we plan to explore cross-task layer sharing (Padmanabhan et al., 2023) and broader framework support in future work. Detailed discussions are provided in Appendix D.

8. Conclusion

We present RTInfer, a system that enables high-density real-time DNN inference on edge GPUs through effective concurrency. RTInfer combines accuracy-calibrated lightweight variant co-optimization in the offline stage with memory-layout-aware task scheduling and a load-aware Delta-Graph pipeline at runtime, reducing memory and transfer overheads while supporting real-time variant switching. Experiments demonstrate that RTInfer consistently outperforms state-of-the-art methods in both deadline miss ratio and accuracy.

Acknowledgement

We sincerely thank the anonymous reviewers for their helpful comments to improve the quality of this paper. This work is supported by the Major Program of Zhejiang NSF Grant No. LD26F020002, the National Natural Science Foundation of China under Grant No. 92582114, No. 625B2167, and No. 62272407.

Impact Statement

This paper presents RTInfer, a system for real-time DNN inference on edge devices. By enabling concurrent, accuracy-calibrated inference, RTInfer may have several societal impacts:

- **Safety-Critical Systems:** The primary impact lies in enhancing the reliability and responsiveness of edge-based applications such as autonomous vehicles and unmanned aerial vehicles. By significantly reducing the deadline miss rate while maintaining high inference accuracy, RTInfer contributes to the operational safety of these systems, potentially reducing risks associated with computational delays in obstacle detection or navigation.
- **Energy Efficiency and Sustainability:** Our evaluation demonstrates that RTInfer provides a high “return on investment” for energy consumption. By optimizing GPU resource utilization and enabling efficient multi-tasking on low-power edge hardware, this work supports the development of more sustainable AI deployments, reducing the overall carbon footprint of large-scale distributed edge intelligence.
- **Accessibility of AI:** By overcoming hardware resource constraints (e.g., limited GPU memory) through intelligent scheduling and model pruning, our techniques make advanced, high-accuracy deep learning models more accessible for deployment on affordable, resource-constrained edge devices.

We do not foresee any specific negative ethical or societal impacts that are unique to this work beyond those generally associated with the advancement of autonomous systems and edge computing.

References

- Ahmed, M. M., Amjad, M., Qureshi, M. A., Khan, M. O., and Haider, Z. M. Optimal pmu placement to enhance observability in transmission networks using ilp and degree of centrality. *Energies*, 17(9):2140, 2024.
- Bolukbasi, T., Wang, J., Dekel, O., and Saligrama, V. Adaptive neural networks for efficient inference. In *Inter-*

- national conference on machine learning*, pp. 527–536. PMLR, 2017.
- Chen, Z., Fang, G., Ma, X., and Wang, X. 0.1% data makes segment anything slim. *arXiv preprint arXiv:2312.05284*, 2023.
- Dehghankar, M., Erfanian, M., and Asudeh, A. An efficient matrix multiplication algorithm for accelerating inference in binary and ternary neural networks. In *Forty-second International Conference on Machine Learning*.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Fang, G., Ma, X., Song, M., Mi, M. B., and Wang, X. Depgraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16091–16101, 2023.
- Fang, G., Ma, X. T., Mi, M. B., and Wang, X. Isomorphic pruning for vision models. *arXiv preprint arXiv:2407.04616*, 2024a.
- Fang, G., Yin, H., Muralidharan, S., Heinrich, G., Pool, J., Kautz, J., Molchanov, P., and Wang, X. Maskllm: Learnable semi-structured sparsity for large language models. *Advances in Neural Information Processing Systems*, 37:7736–7758, 2024b.
- Fei-Fei, L. and Perona, P. A bayesian hierarchical model for learning natural scene categories. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR’05)*, volume 2, pp. 524–531. IEEE, 2005.
- Geiger, A., Lenz, P., and Urtasun, R. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.
- Ghafari, R. and Mansouri, N. An efficient task scheduling in fog computing using improved artificial hummingbird algorithm. *Journal of Computational Science*, 74:102152, 2023.
- Gou, J., Yu, B., Maybank, S. J., and Tao, D. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6):1789–1819, 2021.
- Han, L., Zhou, Z., and Li, Z. Pantheon: Preemptible multi-dnn inference on mobile edge gpus. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services (MobiSys’24)*, pp. 465–478, 2024.
- Han, M., Zhang, H., Chen, R., and Chen, H. Microsecond-scale preemption for concurrent gpu-accelerated dnn inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI’22)*, pp. 539–558, 2022.
- Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.
- Huang, C.-C., Jin, G., and Li, J. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS’20)*, pp. 1341–1355, 2020.
- Jang, W., Jeong, H., Kang, K., Dutt, N., and Kim, J.-C. R-tod: Real-time object detector with minimized end-to-end delay for autonomous driving. In *2020 IEEE Real-Time Systems Symposium (RTSS’20)*, pp. 191–204, 2020.
- Jia, F., Zhang, D., Cao, T., Jiang, S., Liu, Y., Ren, J., and Zhang, Y. Codl: efficient cpu-gpu co-execution for deep learning inference on mobile devices. In *MobiSys*, volume 22, pp. 209–221, 2022.
- Jocher, G., Qiu, J., and Chaurasia, A. Ultralytics yolo, 2023. URL <https://github.com/ultralytics/ultralytics>. Software.
- Kang, W., Lee, K., Lee, J., Shin, I., and Chwa, H. S. Lalarand: Flexible layer-by-layer cpu/gpu scheduling for real-time dnn tasks. In *In 2021 IEEE Real-Time Systems Symposium (RTSS’21)*, 2021.
- Kaya, Y., Hong, S., and Dumitras, T. Shallow-deep networks: Understanding and mitigating network overthinking. In *International conference on machine learning*, pp. 3301–3310. PMLR, 2019.
- Khani, M., Ananthanarayanan, G., Hsieh, K., Jiang, J., Ne-travali, R., Shu, Y., Alizadeh, M., and Bahl, V. RECL: Responsive Resource-Efficient continuous learning for video analytics. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI’23)*, pp. 917–932. USENIX Association, April 2023.
- Kim, S., Genc, H., Nikiforov, V. V., Asanović, K., Nikolić, B., and Shao, Y. S. Moca: Memory-centric, adaptive execution for multi-tenant deep neural networks. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 828–841. IEEE, 2023.

- Kong, R., Li, Y., Feng, Q., Wang, W., Ye, X., Ouyang, Y., Kong, L., and Liu, Y. Swapmoe: Serving off-the-shelf moe-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL'24)*, pp. 6710–6720, 2024.
- Li, X., Li, Y., Li, Y., Cao, T., and Liu, Y. Flexnn: Efficient and adaptive dnn inference on memory-constrained edge devices. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom'24)*, pp. 709–723, 2024.
- Ling, N., Wang, K., He, Y., Xing, G., and Xie, D. Rt-mdl: Supporting real-time mixed deep learning tasks on edge platforms. In *Proceedings of the 19th ACM conference on embedded networked sensor systems (SenSys'21)*, pp. 1–14, 2021.
- Liu, S., Guo, B., Fang, C., Wang, Z., Luo, S., Zhou, Z., and Yu, Z. Enabling resource-efficient aiot system with cross-level optimization: A survey. *IEEE Communications Surveys & Tutorials*, 2023.
- Lu, G., Li, Z., Sun, H., Yu, X., Qin, J., Qiu, J., and Gao, H. A two-phase pcba optimization with ilp model and heuristic for a beam head placement machine. *IEEE Transactions on Industrial Informatics*, 2024.
- Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems (NeurIPS'23)*, 36: 21702–21720, 2023a.
- Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems*, 2023b.
- Ma, X., Fang, G., and Wang, X. Deepcache: Accelerating diffusion models for free. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- Maas, M., Beaugnon, U., Chauhan, A., and Ilbeyi, B. Telamalloc: Efficient on-chip memory allocation for production machine learning accelerators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'23)*, pp. 123–137, 2023.
- Mehta, S. and Rastegari, M. Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*, 2021.
- NVIDIA. Jetson modules, 2024. URL <https://developer.nvidia.com/embedded/jetson-modules>.
- Padmanabhan, A., Agarwal, N., Iyer, A., Ananthanarayanan, G., Shu, Y., Karianakis, N., Xu, G. H., and Netravali, R. Gemel: Model merging for {Memory-Efficient},{Real-Time} video analytics at the edge. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI'23)*, pp. 973–994, 2023.
- Peng, X., Shi, X., Dai, H., Jin, H., Ma, W., Xiong, Q., Yang, F., and Qian, X. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'20)*, pp. 891–905, 2020.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Ramesh, J., Solatidehkordi, Z., El-Fakih, K., and Aburukba, R. Minimizing virtual machine live migration latency for proactive fault tolerance using an ilp model with hybrid genetic and simulated annealing algorithms. *IEEE Access*, 2024.
- Singh, P., Verma, V. K., Rai, P., and Namboodiri, V. P. Play and prune: adaptive filter pruning for deep model compression. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pp. 3460–3466, 2019.
- Wan, A., Hao, H., Patnaik, K., Xu, Y., Hadad, O., Güera, D., Ren, Z., and Shan, Q. Upscale: unconstrained channel pruning. In *International Conference on Machine Learning*, pp. 35384–35412. PMLR, 2023.
- Wang, Q., Xu, M., Jin, C., Dong, X., Yuan, J., Jin, X., Huang, G., Liu, Y., and Liu, X. Melon: Breaking the memory wall for resource-efficient on-device machine learning. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys'22)*, pp. 450–463, 2022.
- Wei, J., Cao, T., Cao, S., Jiang, S., Fu, S., Yang, M., Zhang, Y., and Liu, Y. Nn-stretch: Automatic neural network branching for parallel inference on heterogeneous multi-processors. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*, pp. 70–83, 2023.
- Xiang, Y. and Kim, H. Pipelined data-parallel cpu/gpu scheduling for multi-dnn real-time inference. In *2019 IEEE Real-Time Systems Symposium (RTSS'19)*, pp. 392–405. IEEE, 2019.
- Xin, J., Tang, R., Lee, J., Yu, Y., and Lin, J. Deebert: Dynamic early exiting for accelerating bert inference. *arXiv preprint arXiv:2004.12993*, 2020.

- Xu, G., Hao, J., Shen, L., Hu, H., Luo, Y., Lin, H., and Shen, J. Lgvit: Dynamic early exiting for accelerating vision transformer. In *Proceedings of the 31st ACM International Conference on Multimedia (MM'23)*, pp. 9103–9114, 2023.
- Xu, M., Liu, J., Liu, Y., Lin, F. X., Liu, Y., and Liu, X. A first look at deep learning apps on smartphones. In *The World Wide Web Conference (WWW'19)*, pp. 2125–2136, 2019.
- Yi, J. and Lee, Y. Heimdall: mobile gpu coordination platform for augmented reality applications. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, pp. 1–14, 2020.
- Yu, F., Huang, K., Wang, M., Cheng, Y., Chu, W., and Cui, L. Width & depth pruning for vision transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI'22)*, volume 36, pp. 3143–3151, 2022.
- Zhao, Z., Wang, K., Ling, N., and Xing, G. Edgeml: An automl framework for real-time deep learning on the edge. In *Proceedings of the international conference on internet-of-things design and implementation (IoTDI'21)*, pp. 133–144, 2021.
- Zhao, Z., Ling, N., Guan, N., and Xing, G. Miriam: Exploiting elastic kernels for real-time multi-dnn inference on edge gpu. In *Proceedings of the 21st ACM Conference on Embedded Networked Sensor Systems (SenSys'23)*, pp. 97–110, 2023.

A. Extended System Design Details and Algorithmic Implementation

A.1. Pruning Challenges and Multi-step Logic

Extensive pruning works (Fang et al., 2024b; 2023; 2024a; Ma et al., 2023b; 2024; Chen et al., 2023; Singh et al., 2019) have focused on efficient and adaptive model pruning, including selectively trimming network weights while considering the importance of each layer and the complex inter-dependencies within the network.

However, applying pruning methods to early exit mechanisms straightforwardly presents a series of challenges. Firstly, all inserted early exits also require pruning. Since these early exits are part of the network structure, they can be considered as output layers that output intermediate results prematurely, and their layer-to-layer complex dependencies must be considered in conjunction with the preceding connected structures. Secondly, because early exits are inserted into chunks composed after model partitioning, the pruning operation must not disrupt the effective transmission of intermediate results between chunks. Thirdly, the complex dependencies between the early exits and the multiple blocks, as well as the importance of the weights, also need to be effectively assessed to ensure that important parts of the network are not pruned, which could lead to a sudden drop in accuracy. To address this, we propose an accuracy-calibrated lightweight variant co-optimization method that effectively tackles the aforementioned issues.

Next, we will provide a detailed introduction to pruning for DNN chunks and early exits between chunks. A schematic diagram of our pruning method is shown in Figure 6.

Step1: When pruning Chunk 1 and Exit 1, during this particular step, we only need to consider the dependency between them and the importance of the weights (Fang et al., 2023). Special attention is paid to the structure of the last layer of Chunk 1 and the first layer of Exit 1 to ensure that pruning does not affect the transmission of intermediate results from Chunk 1 to Exit 1.

Step2: When pruning Chunk 2 and Exit 2, it is imperative to first ensure that the intermediate results from Chunk 1 can be successfully passed to Chunk 2. To prevent modifications to the structure of the initial block that would make it difficult for subsequent chunks to use the intermediate outputs from Chunk 1 as inputs, we freeze the weight parameters of Chunk 1, prohibiting them from participating in subsequent pruning processes. Furthermore, we analyze the dependencies between Chunk 1, Chunk 2, and Exit 2, and assess the importance of weights within both the chunks and the exit points. During the pruning process, it is crucial to efficiently identify critical paths and effectively avoid them to prevent irreversible impacts on the accuracy of the results.

Algorithm 1 Heuristic Buffer Allocation for ILP

Input : Buffer set $\mathcal{B}$, memory budget M , maximum search steps $S_{\max}$

Output : A feasible memory layout $\mathcal{S}$ for buffers, or failure

Initialize $pos[b]$ for each buffer $b \in \mathcal{B}$ Initialize $steps \leftarrow 0$
 $Q \leftarrow$ priority queue of buffers sorted by lifetime, size, and area
 $\mathcal{S} \leftarrow \emptyset$

while $Q \neq \emptyset$ **and** $steps < S_{\max}$ **do**

$b \leftarrow Q.pop()$

if $pos[b] + size[b] > M$ **or not** $VALIDPLACEMENT(b, pos, \mathcal{S})$ **then**

$\mathcal{C} \leftarrow IDENTIFYCONFLICTS(b, pos, \mathcal{S})$

if $ISMINORCONFLICT(\mathcal{C})$ **then**

$MINORBACKTRACK(\mathcal{C}, Q, pos)$

else

$MAJORBACKTRACK(\mathcal{C}, Q, pos)$

$steps \leftarrow steps + 1$ **continue**

$\mathcal{S} \leftarrow \mathcal{S} \cup \{b\}$ $UPDATEPOSITIONS(b, pos, \mathcal{S})$ $steps \leftarrow steps + 1$

if $Q = \emptyset$ **then**

return $\mathcal{S}$

else

return FAILURE

Step3: The pruning process for this and subsequent steps is essentially consistent with Step 2, continuing until all model structures have been traversed. Additionally, pruning is only allowed for the last chunk within the gray box of Step 3, with all chunks preceding it being frozen.

A.2. Comparative Analysis of Optimization Solvers

We justify our mixed-variable GA against three common alternatives: 1. **Grid Search:** Intractable. With $|P|$ continuous pruning ratios and $|E|$ discrete exits, the space $\mathbb{R}^{|P|} \times \{0, 1\}^{|E|}$ is infinite. 2. **Bayesian Optimization (BO):** BO’s Gaussian Process kernel assumes continuity. Memory layout feasibility (C_{mem}) creates “cliffs” where a 1KB change makes a layout impossible, causing BO to fail. 3. **ILP with Linear Proxies:** Accuracy loss is highly non-linear ($\approx 1/x^2$). Linearizing Eq. 1 introduces $> 25\%$ modeling error.

A.3. Heuristic-driven Search and Backtracking Logic

We use a standard Integer Linear Program (ILP) (Lu et al., 2024; Ramesh et al., 2024; Ahmed et al., 2024) solver, which encodes it as Boolean variables, i.e., $B_{i,j} \vee \tilde{B}_{i,j}$.

In each subsequent step, we make selections from a set of heuristic methods that choose the next buffer block of the variant’s chunk to be placed and determine its location. We refer to these blocks as candidate blocks. This effectively simulates a search tree, where each node (or decision point) represents a state (i.e., a set of already placed blocks and their positions). The depth of a node is referred to as the decision level of the decision point.

Our task scheduling algorithm operates within a two-dimensional space defined by Address and Time. Each DNN task variant’s chunk requires specific memory resources, represented by rectangular buffers. The latency of a chunk determines the buffer’s time length, and the memory requirements of the variant dictate the height of the buffer in the Address dimension. The objective is to place these buffer blocks without overlap, ensuring that the total memory usage at any point does not exceed the device’s available memory.

We show our heuristic algorithm in Algorithm 1. At each step, the strategy considers all unplaced blocks and selects the next one using three heuristic methods: (1) The block with the longest lifetime (end-start time), (2) The block with the largest size, (3) The block with the largest “area” (size $\times$ lifetime). These heuristics are attempted in sequence at each step. The algorithm initially attempts the longest allocation, potentially impacting the most constraints, and follows this path in the search tree. If this fails, it tries the largest allocation within 10% of the single task inference time, continuing until exceeding the time limit or exhausting options, necessitating backtracking. By running the solver at each step, infeasible sections of the search tree are quickly bypassed. Two types of backtracking are employed: minor and major. Minor backtracking occurs when a block placement immediately makes memory allocation infeasible, returning to the previous step to choose the next candidate block. Major backtracking is required when all candidate blocks at a decision point are exhausted, returning to the second-to-last conflict.

After each placement step, RTInfer checks whether the selected variant chunk can still finish before the task deadline. If the remaining execution would violate the deadline, RTInfer stops executing the current and subsequent chunks and triggers the earliest feasible exit to return a result before the deadline.

B. Detailed Implementation

B.1. Offline Variant Generation

We construct a bank of DNN variants per task by crossing structured channel pruning with early-exit heads and fine-tuning. Each variant is profiled for latency, peak memory, and accuracy before being converted to TorchScript. This retains a compact, accuracy-preserving set of runtime choices for the online scheduler.

B.2. Allocator Triggers and Eviction Policy

The allocator is invoked only when (i) admitting a new RT job, (ii) a variant switch elevates memory demand, or (iii) fragmentation exceeds a threshold ($\geq 12\%$ slack loss). Otherwise, the last layout is reused. This hybrid policy amortizes

allocation costs while keeping fragmentation bounded and preserving the scheduler’s sub-millisecond budget.

B.3. Admission and Runtime Telemetry

Upon RT arrivals or elevated deadline risk, low-priority BE streams are preempted and rescheduled. Runtime telemetry is sampled every 5 ms via CUPTI on a pinned CPU thread ($\leq 0.2\%$ CPU). Variant selection is $O(1)$ and adds $<50\mu\text{s}$ per job, keeping deadlines feasible before resorting to earlier exits.

B.4. Scope and Pipeline Graphs

The runtime supports pipeline graphs by treating each DNN as a node with ring-buffer communication; scheduling operates at job boundaries over node sequences. We do not cross-task reuse intermediate activations in the current implementation to favor isolation, while leaving room for future memory sharing.

C. Extended In-depth Analysis

C.1. Variant Design Space Exploration

Figure 14 illustrates the accuracy–latency–memory trade-offs across pruning ratios and early-exit depths. The latency and memory footprint are profiled on Jetson Xavier NX, while accuracy is computed using the calibrated ALC variant model. As pruning ratio increases from 0 to 0.75, the full-exit variant reduces latency from 10.68 ms to 5.70 ms and memory footprint from 15.87 MiB to 5.40 MiB, with calibrated accuracy decreasing moderately from 0.965 to 0.921. Deeper exits provide higher accuracy at the cost of higher resource usage, whereas pruning substantially improves compactness and latency. This design space allows RTInfer to select variants that preserve deeper exits under resource pressure, instead of falling back aggressively to shallow exits. Minor non-monotonic latency variations arise from exit-head and CUDA kernel efficiency, but the overall trend confirms that pruning and early exits provide a practical accuracy–efficiency trade-off.

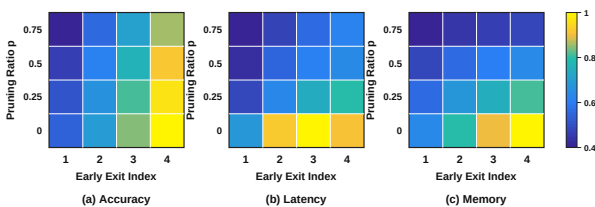


Figure 14. Impact of pruning ratio and early-exit depth on accuracy, latency, and memory footprint.

C.2. Detailed Ablation Analysis

To further characterize the contribution of each component, we define the following ablation variants:

- **RTInfer-w/o-ALC**: Disables the Accuracy-calibrated Lightweight Variant Co-optimization. This version handles tasks with a single, fixed model, leading to parallel execution failure under heavy load.
- **RTInfer-w/o-MS**: Disables the Memory-layout-aware Scheduler. Inefficient memory management leads to higher fragmentation, breaking the bounded-memory execution guarantees.
- **RTInfer-w/o-DLP**: Disables the Delta-Graph and load-aware Pipeline. This version transfers full parameter blocks on each variant switch, making the switching delay unpredictable.

D. Extended Discussion

Supported models. In this paper, we focus on CNN models because they are widely deployed on devices. For example, a large-scale study conducted on 87.7% of DL apps on smartphones revealed that the models are predominantly CNNs (Xu et al., 2019). RTInfer is applicable to other neural network tasks, such as transformers. Specifically, early exits in Transformer models have been explored (Xu et al., 2023; Xin et al., 2020), and pruning techniques are well-studied (Yu et al., 2022; Kong et al., 2024; Ma et al., 2023a). Thus, RTInfer’s accuracy-calibrated lightweight variant co-optimization method is still applicable. Furthermore, RTInfer’s memory layout-based scheduling algorithm is agnostic to the specific model architecture and is related to the size of the memory buffer, making it suitable for other DNN architectures as well.

Memory footprint reduction. We find that memory is a major constraint for concurrent multi-DNN inference on edge GPUs, leading us to adopt pruning-based techniques. Other common methods include knowledge distillation (Gou et al., 2021) and model quantization (Han et al., 2015). However, knowledge distillation requires extensive offline processing, which can be computationally expensive. Quantization, on the other hand, maps high-precision floating-point numbers to integers (Han et al., 2015), accelerating inference. Quantization is orthogonal to RTInfer, as RTInfer can still benefit from quantization after pruning to further enhance performance.

Compute-resource contention. RTInfer primarily targets multi-DNN real-time workloads whose bottleneck is device memory pressure, activation footprint, and model-loading overhead. This design does not explicitly schedule SM resources. Therefore, when DNN tasks are dominated by large

CUDA kernels that persistently saturate SMs, or when input scaling significantly increases kernel grid sizes, concurrent execution may still be limited by compute-resource contention even if memory layout is optimized. In such cases, RTInfer can reduce memory footprint and loading delay, but the achievable improvement in deadline satisfaction may be bounded by available SM capacity. Extending RTInfer with SM-aware admission control, kernel throttling, or cooperative CUDA stream scheduling is a promising direction, but it requires lower-level runtime or hardware support and may introduce additional scheduling complexity.

Task Fusion and Future Work. Typically, model differences between various tasks are significant (e.g., traffic light classification vs. obstacle detection). Recent studies ([Khani et al., 2023](#); [Padmanabhan et al., 2023](#)) have proposed fusing different models by sharing common layers to reduce memory overhead, but this adds complexity and assumes high model similarity. RTInfer does not rely on such assumptions. We plan to explore optimization for scenarios with high task similarity and investigate performance across a broader range of devices and frameworks.